

Extraction and OpenRCX

UCSD ECE 260C Spring 2026

Prof. Andrew Kahng

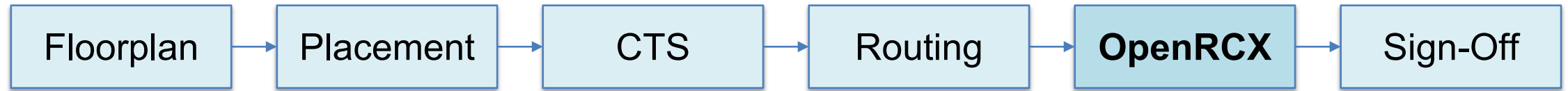
abk@ucsd.edu

Thanks to: David Overhauser and Dimitris Fotakis

Also: Katerina Tsilingiri and Jakang Lee

RC Extraction in the Design Flow

- RC Extraction bridges physical implementation and sign-off analysis
 - OpenRCX transforms routed geometry into electrical models that drive timing, signal integrity and power analysis



- OpenRCX Inputs: DEF (routed geometry), LEF (technology layers), RC Tech File (parasitic models)
- OpenRCX Output: SPEF (Standard Parasitic Exchange Format)

Extraction Purpose

- Parasitic data for performance and signal integrity analysis
 - RC networks for P&R timing and signal integrity analysis (OpenRCX)
 - RC and W for P&R EM analysis
 - RLC, via clusters and W for P&R power grid IR drop and EM analysis
 - Transistor-level simulation and related analyses
- Use Model
 - Full-chip extraction for full-chip analysis
 - Fast RC estimation for pre-routing timing and optimization
 - Incremental ECO extraction for post-route optimization

Provide parasitic and associated data to analysis tools
in the chip design flow

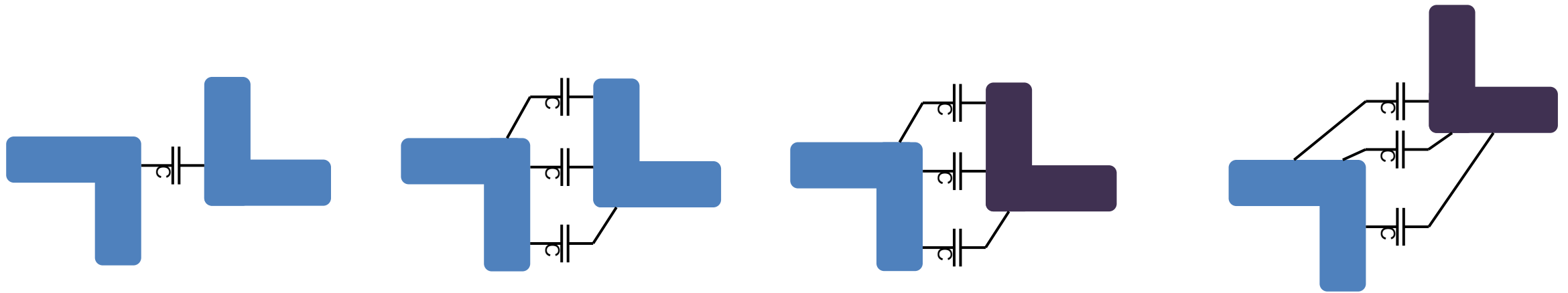
OpenRCX Original Design Purpose

- Tightly coupled with design database (ODB) to support incremental ECO performance
 - Fast geometric, parasitic and timing "do" and "undo"
- Ability to extract multiple corners at once for "scenario" optimization
 - **Scenario:** specific combination of device corner, parasitic corner and setup/hold used in timing analysis (e.g. SS-WRC-setup, FF-BRC-hold, ...)
 - Optimize and evaluate based on multiple scenarios at once
- APIs to timing analysis enable incremental ECO timing
 - Used with OpenSTA and Primetime

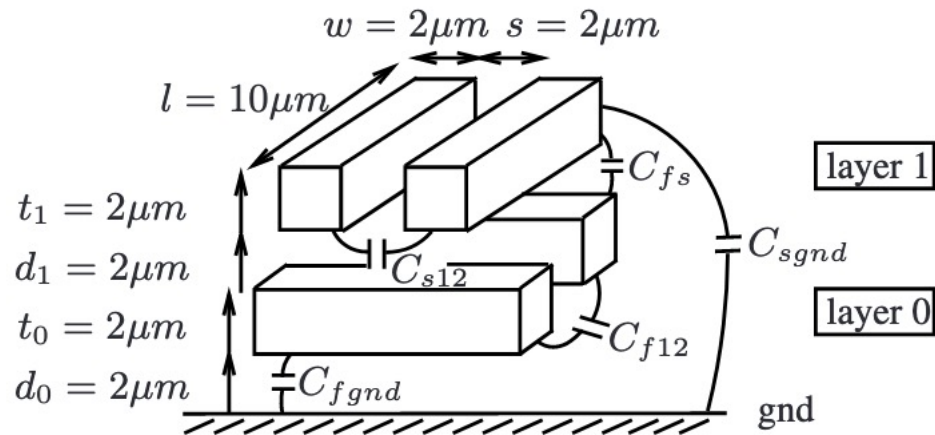
Incremental ECO parasitics for post-route optimization

Extraction Capacitance Model History

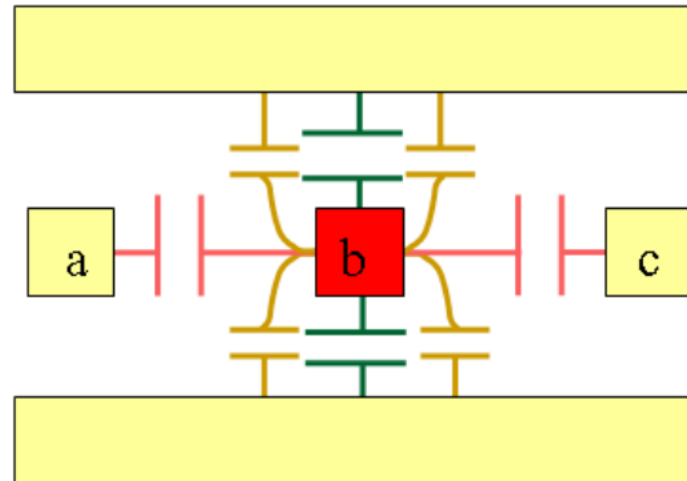
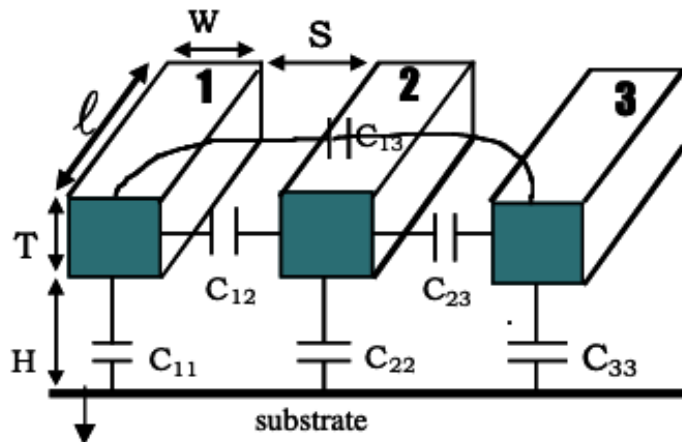
- 3D: field solvers, random walk – true 3D
 - Challenges: performance, mesh generation, non-rectangular shapes
- 1D patterns: extract as parallel plate capacitors – face to face
- 2D patterns: extract patterns on same plane – adding co-planar perpendicular faces (fringe)
- 2.5D patterns: 2D patterns plus models of interactions between non-planar faces
 - Challenges: fracturing, windowing, pattern complexity



Capacitance Extraction Challenges



- Capacitance extraction is a best guess of the electrical parameters of a complex 3D network of structures.
- Pattern-based extractors build a collection of 2.5D models to estimate the various capacitance components in each segment of a wire.
- It is a data intensive 3D geometric operation.



Parallel plate capacitance is on all parallel surfaces.

Fringe capacitance exists to all perpendicular surfaces.

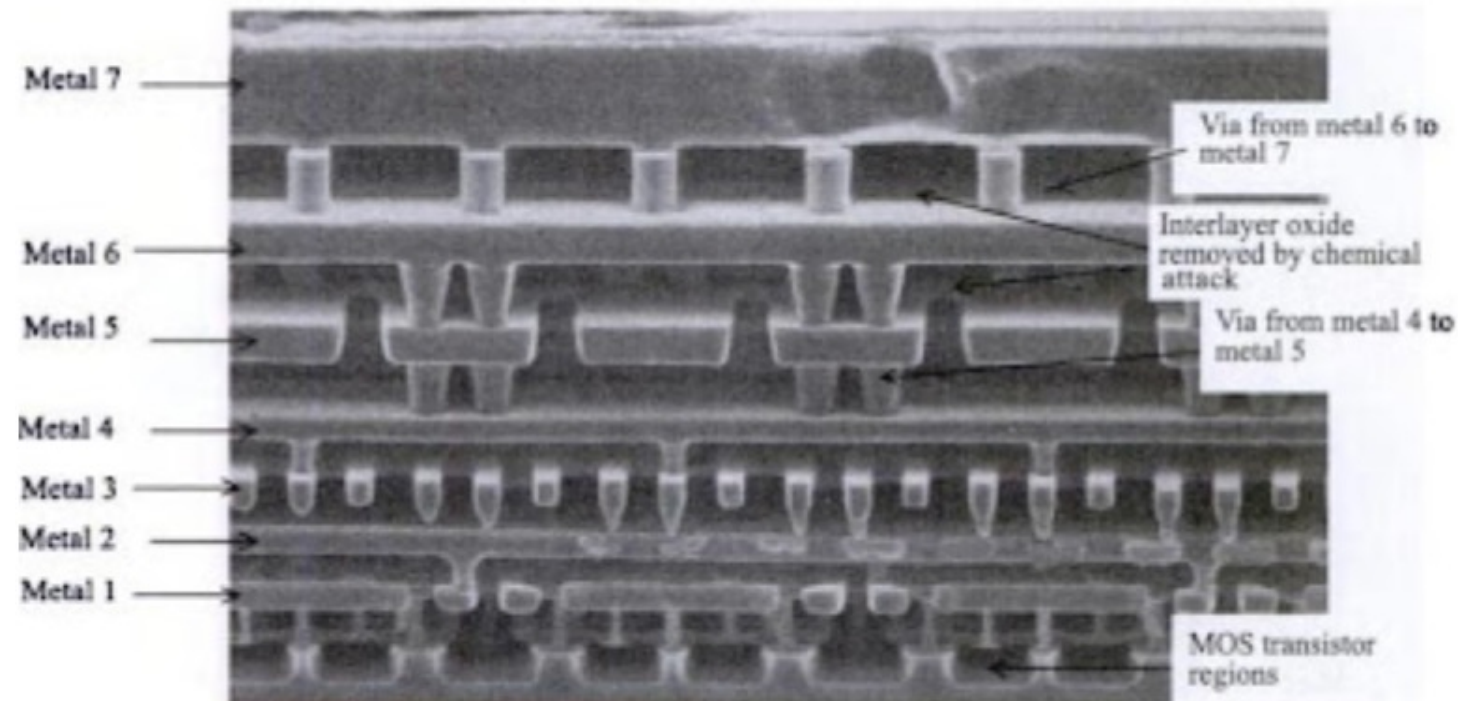
Which dominates depends on spacings.

Sources of RC Extraction Complexity

- Model complexity defined by foundry
- Complex layer stack specification (dielectrics, metals, vias, ...)
- Drawn geometry $\neq$ physical geometry (systematic + statistical)
 - Lithography changes shapes
 - Etching changes cross section
 - CMP changes thickness
- Extractor is a local shape searching engine with models
- R fracturing, RC distribution
- Process corners
- Metal fill

What Makes Extraction Hard

- 7 metal layers, each with its own width, spacing and dielectric and every wire couples to neighbors above, below and on the same layer
- Via resistance is significant and often underestimated
- Wire shapes are not ideal rectangles
- The extractor must model all pairwise interactions across this 3D stack at full-chip scale



Cross-section of a 0.12 μm technology (Courtesy: Fujitsu)

Foundry Table: Resistivity Table in Tech File

- Variation caused by both local and large area systematic process variations (Lithography, CMP)
- Wire thickness modelling generally accounted for after routing is complete (density available)
- Early flow extraction assumes 50% density, or nominal thickness
- Spacing greater than X assumes metal fill added to target 50% density (nominal thickness)

Resistivity = f(Weff, Thickness)										
	Weff/Wmin	Thickness/(nominal thickness)								
		0.7	0.8	0.9	1.0	1.1	1.2	1.3	1.4	1.5
	0.9	0.043	0.043	0.042	0.042	0.042	0.042	0.042	0.041	0.041
	1.0	0.041	0.041	0.040	0.040	0.040	0.040	0.040	0.040	0.039
	1.1	0.041	0.040	0.040	0.040	0.040	0.039	0.039	0.039	0.039
	1.2	0.041	0.041	0.040	0.040	0.040	0.039	0.039	0.039	0.039
	1.3	0.040	0.039	0.039	0.039	0.038	0.038	0.038	0.038	0.038
	1.5	0.038	0.038	0.038	0.037	0.037	0.037	0.037	0.037	0.036
	2.0	0.035	0.035	0.035	0.034	0.034	0.034	0.034	0.034	0.033
	3.0	0.035	0.034	0.034	0.034	0.033	0.033	0.033	0.033	0.033
	4.0	0.035	0.034	0.034	0.034	0.033	0.033	0.033	0.033	0.033
	5.0	0.034	0.034	0.033	0.033	0.033	0.033	0.032	0.032	0.032

Foundry Table: Sheet Resistance (Ohms/square)

Ohms/square = f(Weff, Thickness)										
Weff/Wmin		Thickness/(nominal thickness)								
		0.7	0.8	0.9	1.0	1.1	1.2	1.3	1.4	1.5
	0.9	0.75	0.65	0.57	0.51	0.46	0.42	0.39	0.36	0.33
	1.0	0.71	0.62	0.54	0.49	0.44	0.40	0.37	0.34	0.32
	1.1	0.71	0.61	0.54	0.48	0.44	0.40	0.37	0.34	0.32
	1.2	0.71	0.62	0.54	0.48	0.44	0.40	0.37	0.34	0.32
	1.3	0.69	0.60	0.52	0.47	0.42	0.39	0.35	0.33	0.31
	1.5	0.67	0.58	0.51	0.45	0.41	0.37	0.34	0.32	0.29
	2.0	0.61	0.53	0.46	0.41	0.37	0.34	0.31	0.29	0.27
	3.0	0.60	0.52	0.46	0.41	0.37	0.33	0.31	0.28	0.26
	4.0	0.60	0.52	0.46	0.41	0.37	0.33	0.31	0.28	0.26
	5.0	0.59	0.51	0.45	0.40	0.36	0.33	0.30	0.28	0.26

Foundry Table: Resistance (Ohms/um)

CMP thickness R variation

CMP thickness R variation

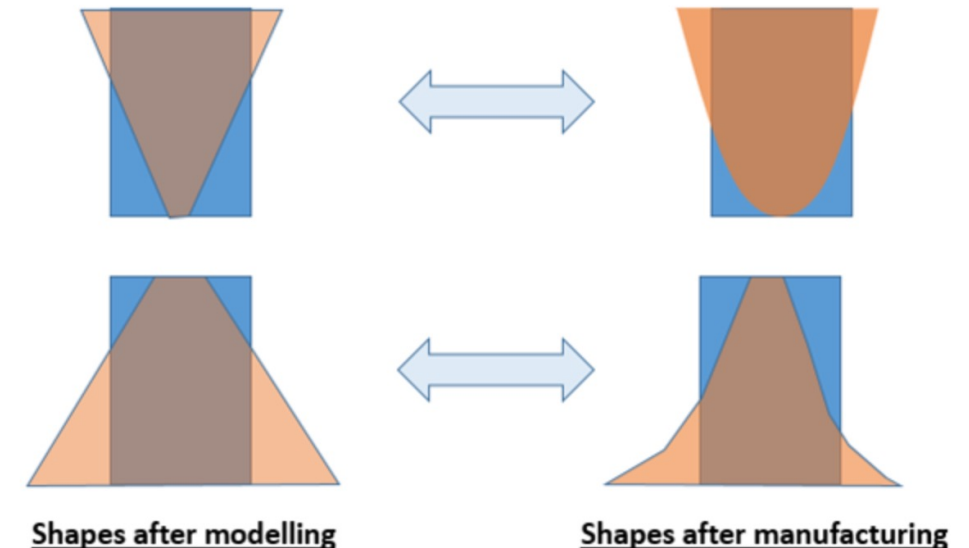
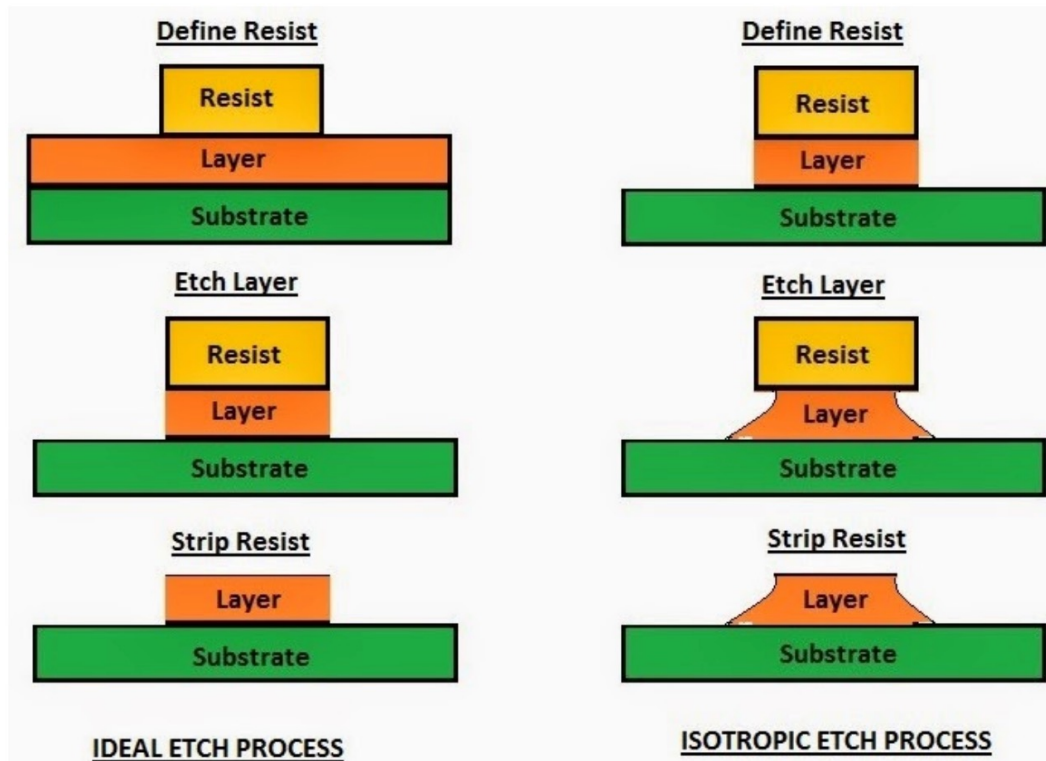
R/um = f(Weff, Thickness)										
Weff/Wmin		Thickness/(nominal thickness)								
		0.7	0.8	0.9	1.0	1.1	1.2	1.3	1.4	1.5
	0.9	19.4	16.8	14.8	13.3	12.0	11.0	10.1	9.3	8.7
	1.0	16.7	14.4	12.7	11.4	10.3	9.4	8.7	8.0	7.5
	1.1	15.1	13.0	11.5	10.3	9.3	8.5	7.8	7.2	6.7
	1.2	13.9	12.0	10.6	9.4	8.5	7.8	7.1	6.6	6.1
	1.3	12.4	10.7	9.4	8.4	7.6	6.9	6.4	5.9	5.5
	1.5	10.4	9.0	7.9	7.1	6.4	5.8	5.3	4.9	4.6
	2.0	7.2	6.2	5.4	4.9	4.4	4.0	3.7	3.4	3.2
	3.0	4.7	4.0	3.6	3.2	2.9	2.6	2.4	2.2	2.1
	4.0	3.5	3.0	2.7	2.4	2.1	2.0	1.8	1.7	1.5
	5.0	2.8	2.4	2.1	1.9	1.7	1.5	1.4	1.3	1.2

Note: via resistance here > 11 Ohms

Increasing width pays off better when > 1.5 x minimum width (e.g. $11.4/2 = 5.7$, but R is 4.9)

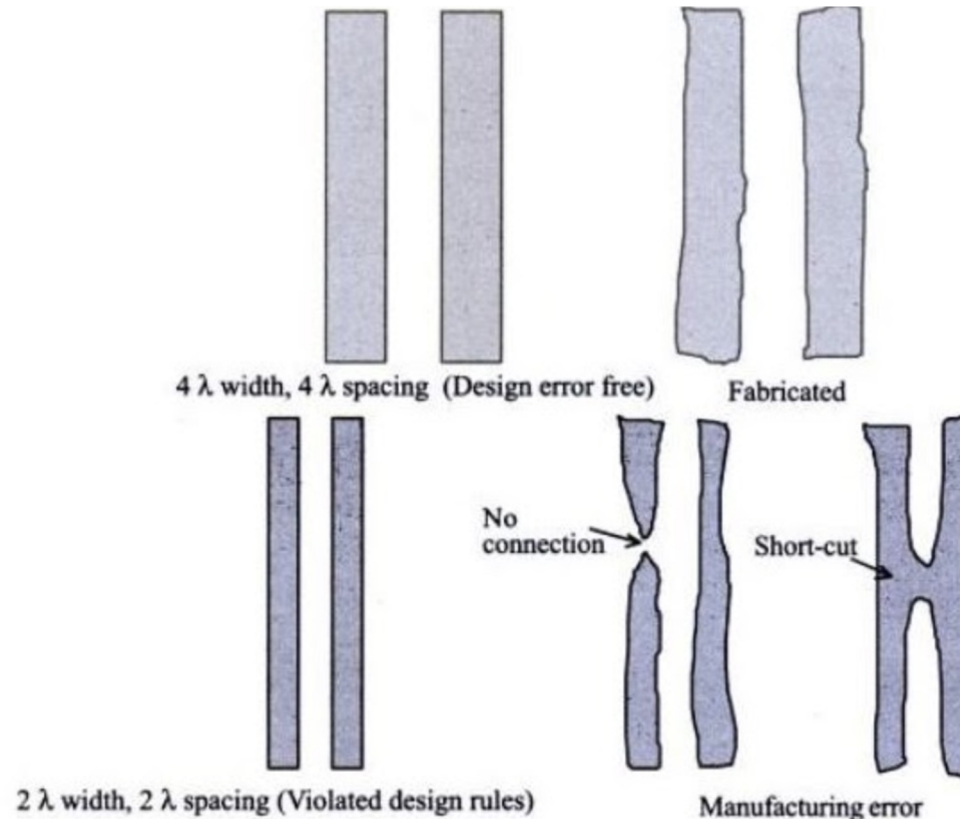
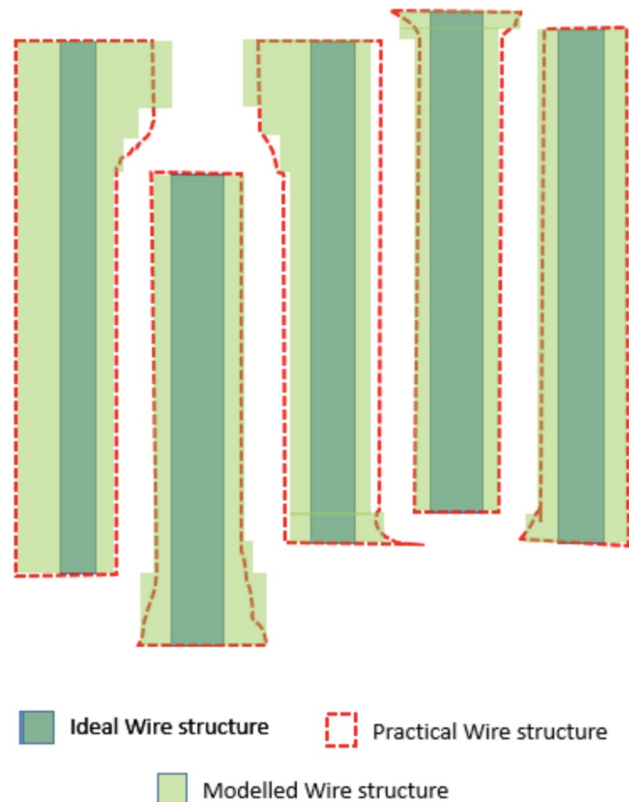
Systematic Wire Width Variation

- During manufacturing lithography and etching wires do not appear as drawn in layout
 - Systematic variations in effective width and shape as a function of drawn wire width/spacing.
- The effects of these systematic variations are modeled when building extraction tech files. “You never get what you draw.”



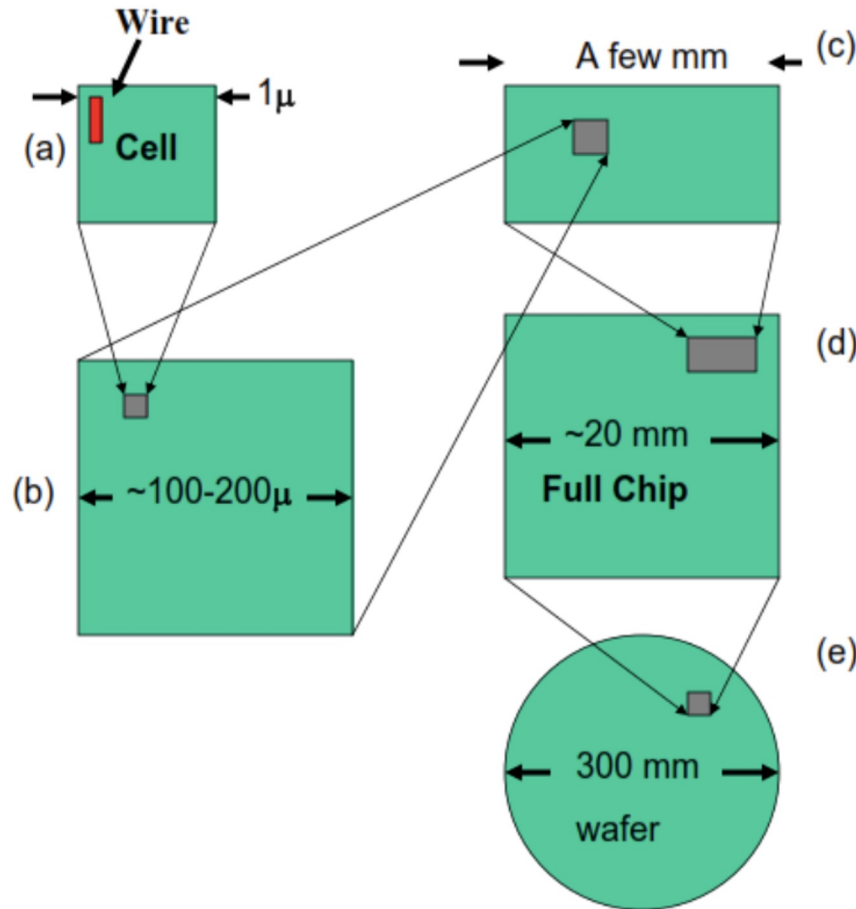
Systematic Wire Width Variation

- It is impracticable for lithography simulation to be applied on every design because there are too many potential nearby structures that impact systematic printing variations.
- The number of potential manufacturing errors caused by systematic lithography variations is addressed using more and complex design rules for chip design.
- New and more complex rules are created for each new process generation.

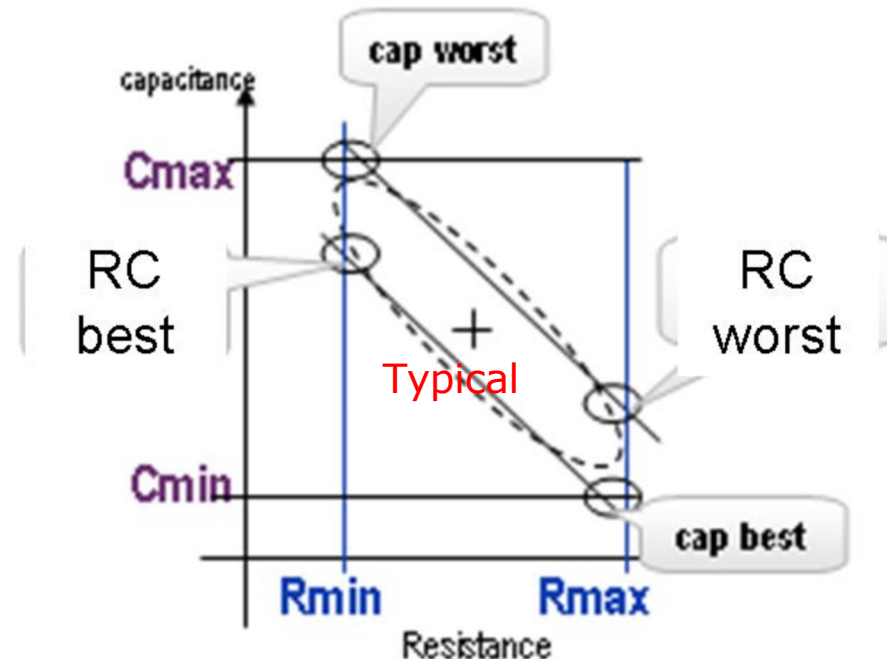


The ability to print a specific pattern can be a function of structures several wires away.

What Are Process Corners



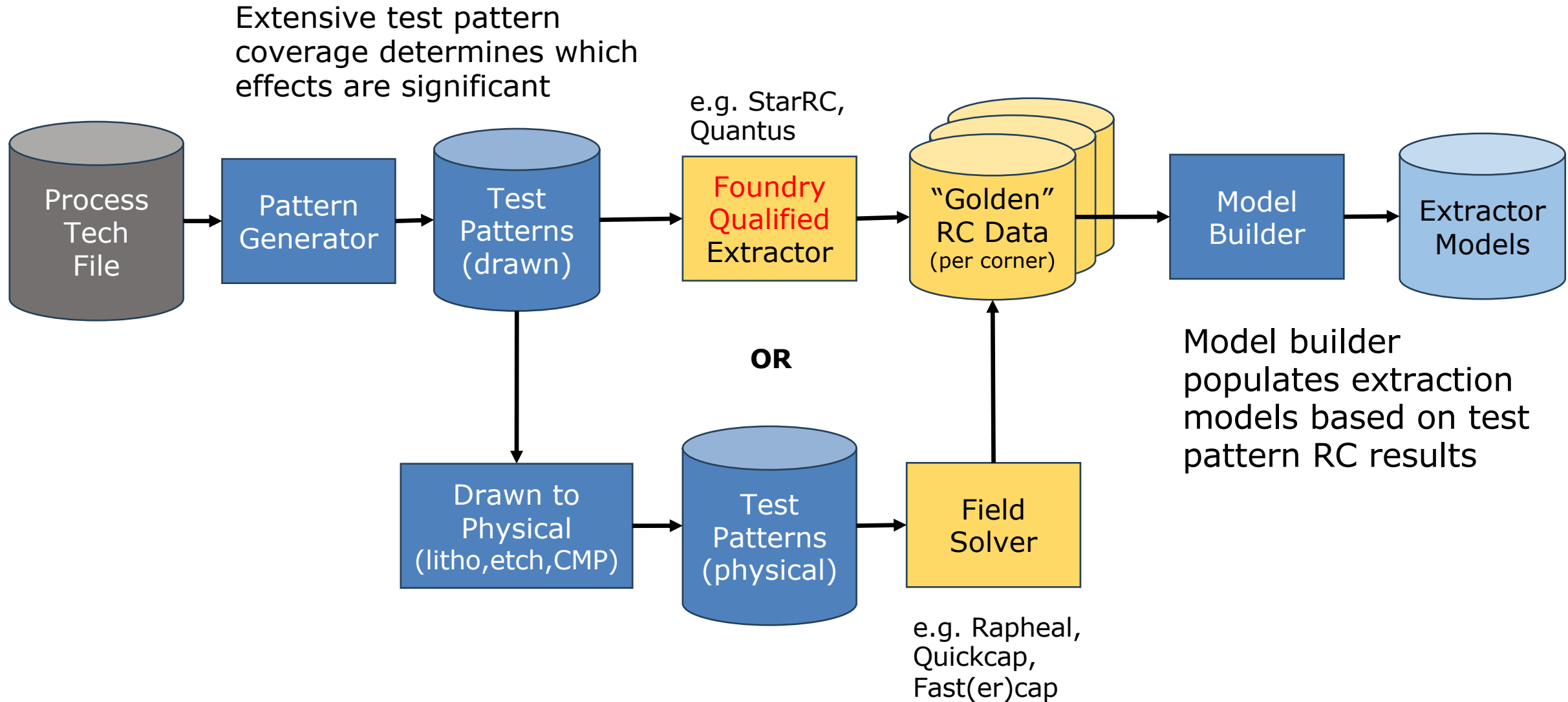
- Manufacturing is not ideal! Variations occur naturally across various distances on a chip and across a wafer.
- Process corners are used to estimate the differences in chip behavior across the wafer.
 - Variations occur in devices as well as wires.
 - Some companies examine dozens of corners.



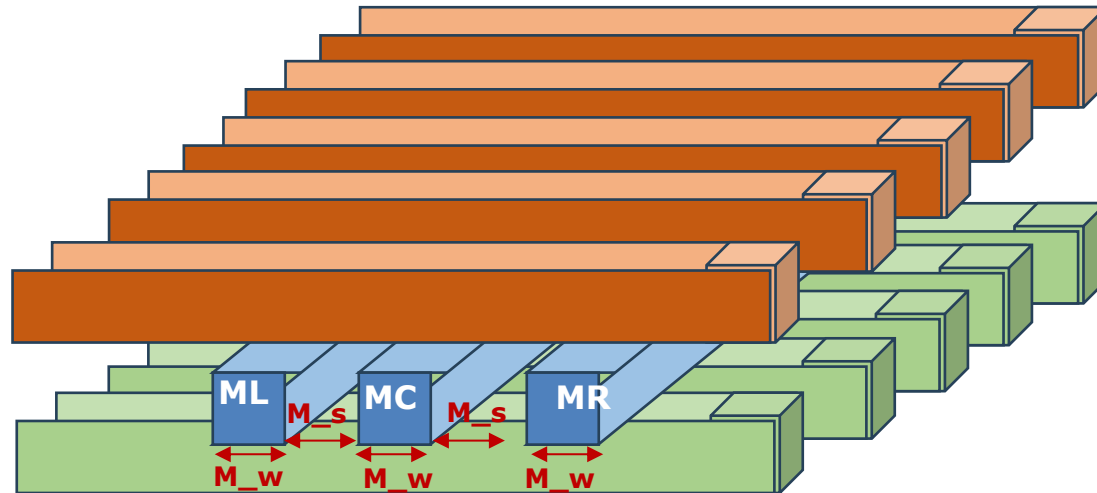
Here are 5 sample process corners just for RC extraction.

Each corner represents a statistical variation in lithography, etch and CMP.

Building Extractor Models



OpenRCX Pattern D



Measure

CC_MC_ML

CC_MC_MR

CC_MC_Mover

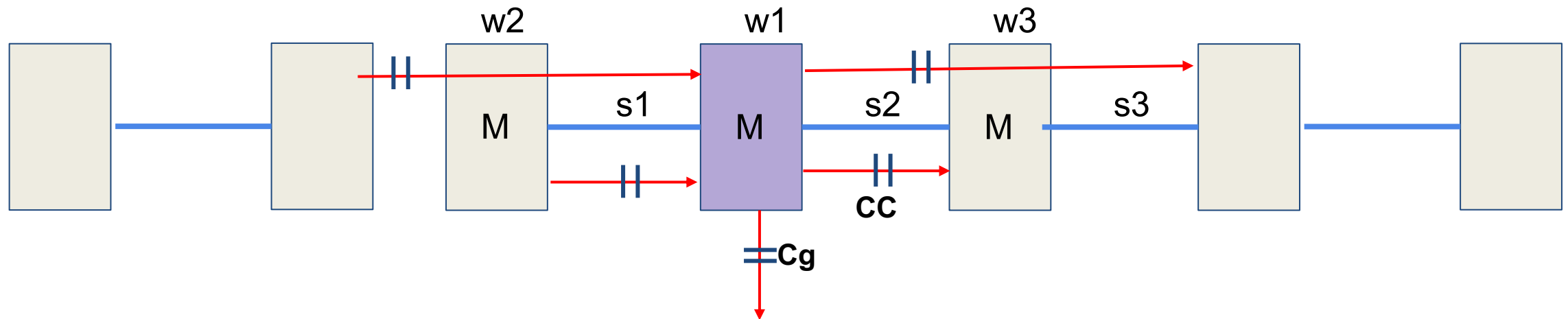
CC_MC_Munder

CC_MC_gnd

Coupling Cap = $f(M_w, M_s, M, \text{Mover}, \text{Munder})$

Grounded Cap = $f(M_w, M_s, M, \text{Mover}, \text{Munder})$

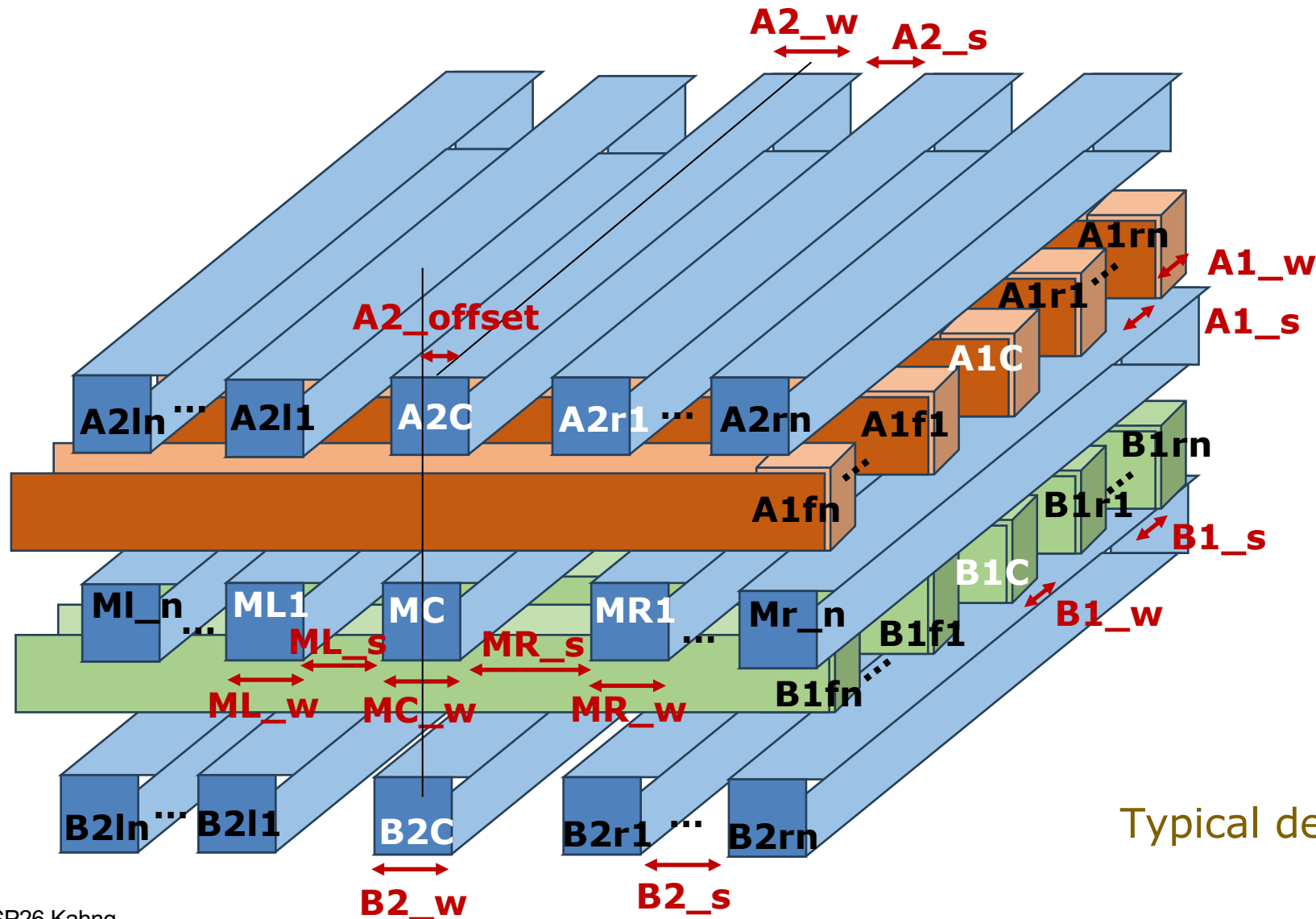
Non-Symmetric Coupling Caps to First and Second Neighbors



Coupling Cap= function(w , s , M), height is set

Fringe (Ground) Cap= function(w , s , M)

Complex Patterns to Characterize Models



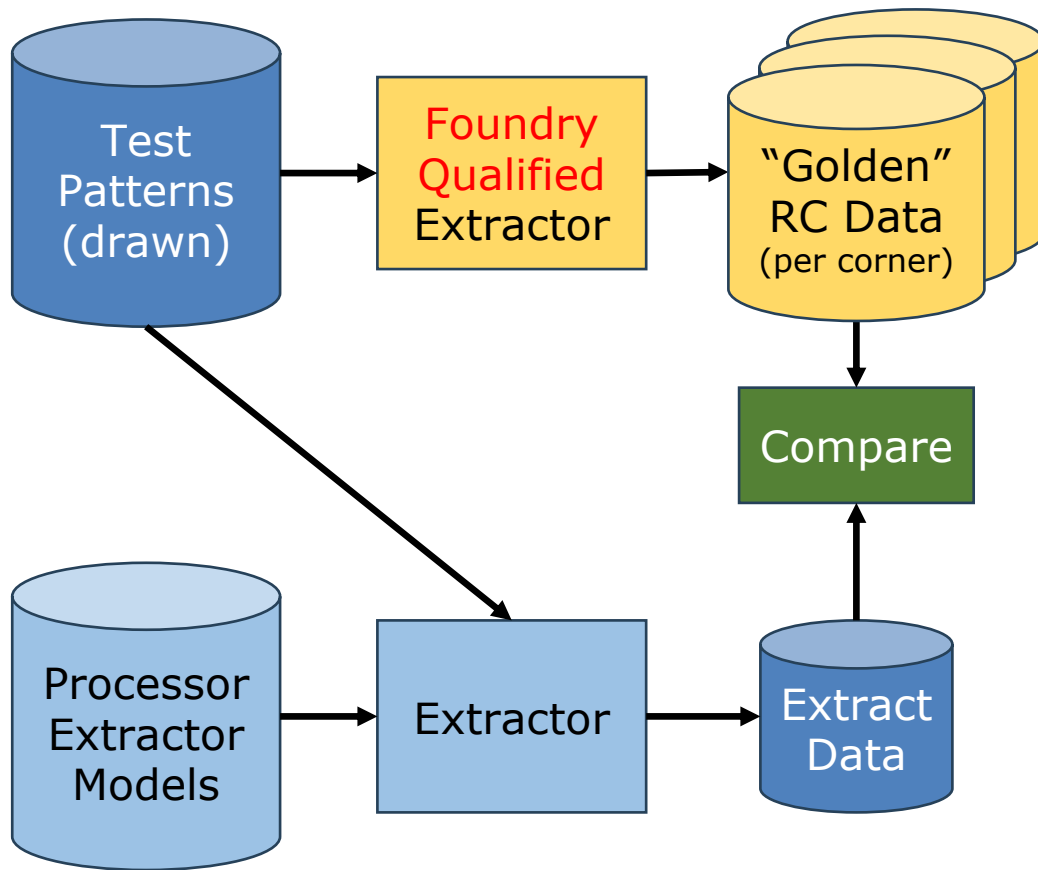
Measure

CC_MC_ML1
 CC_MC_MR1
 CC_MC_A1C
 CC_MC_A2C
 CC_MC_B1C
 CC_MC_B2C
 CC_A1C_B1C
 CC_MC_A2r1

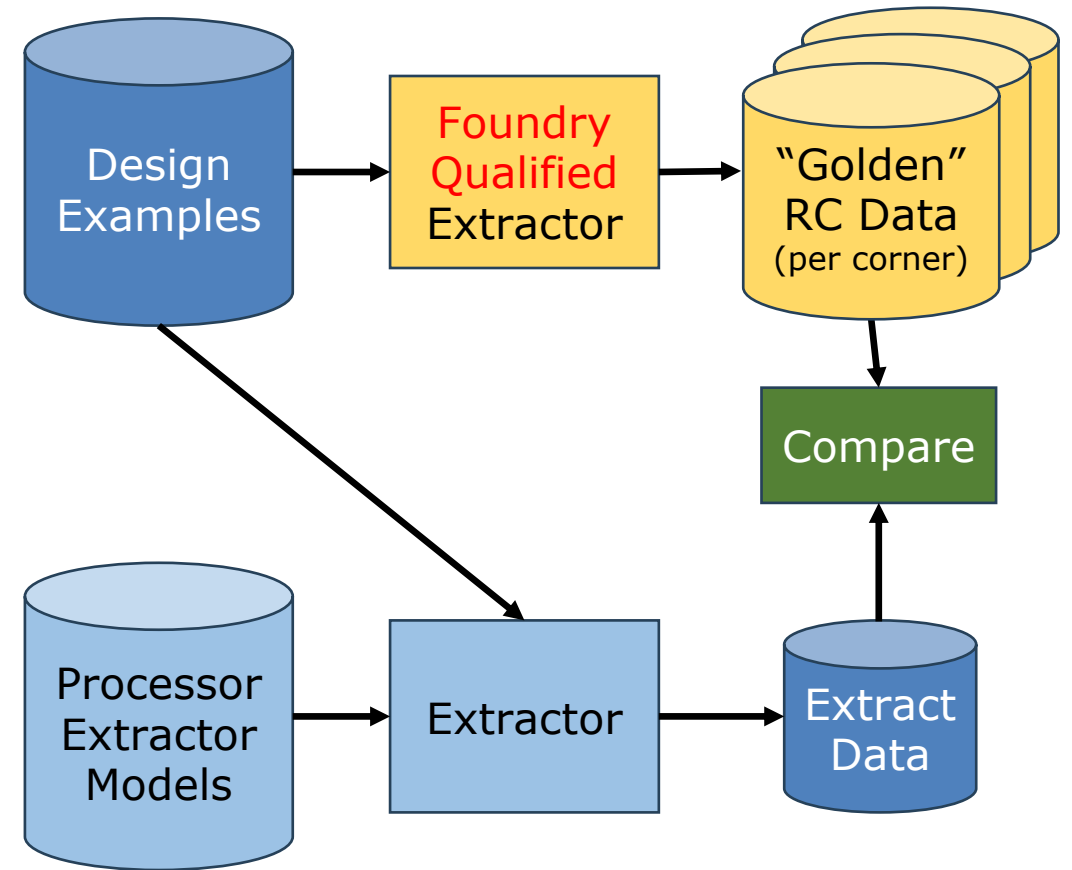
R_MC

Typical density area (50um)²

Verifying Models



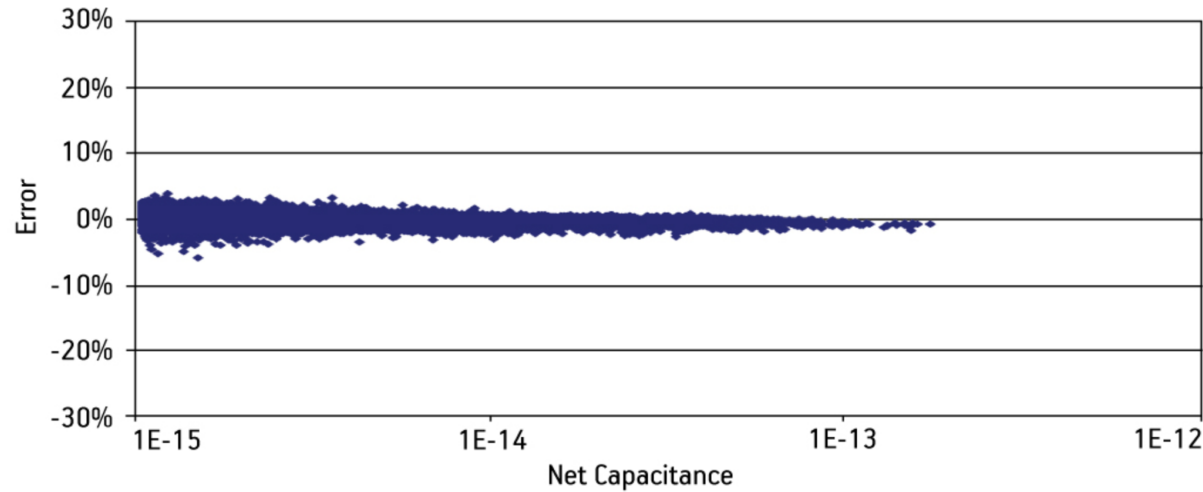
Verify using test patterns (once per process)



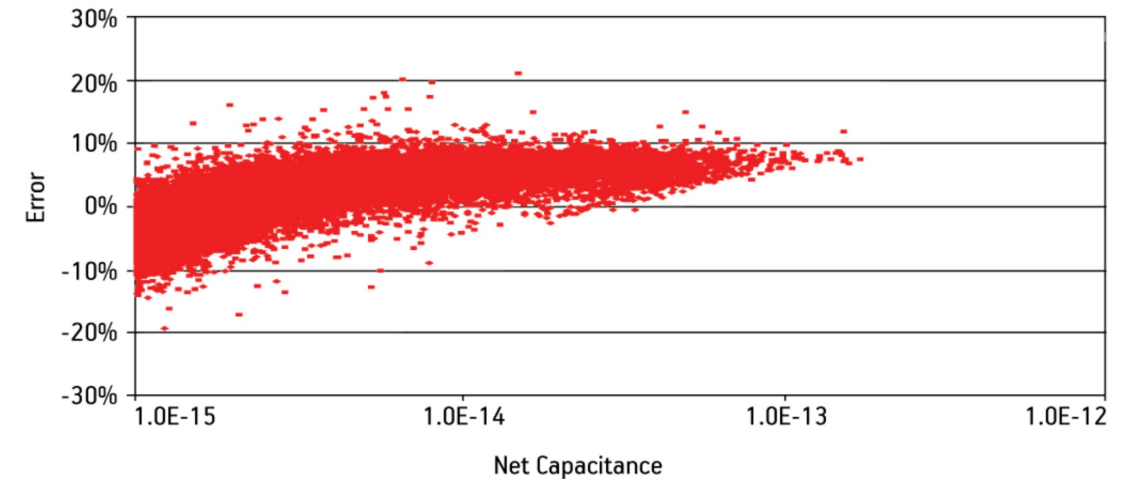
Verify using sample designs

Typical Extraction Verification Results

Error of Field Solver

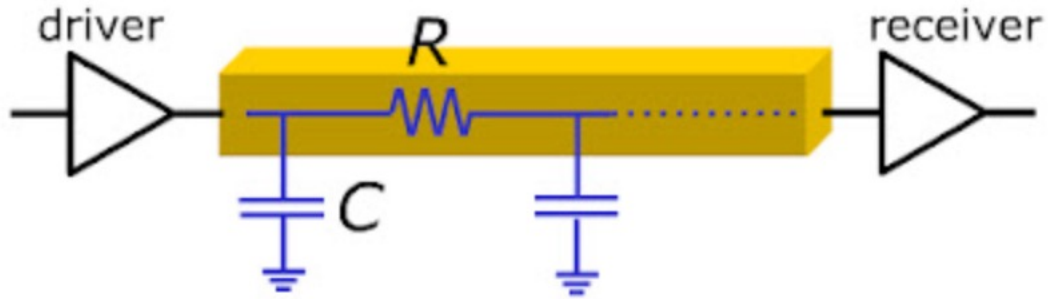


Error of Typical Pattern Matching LPE Tools

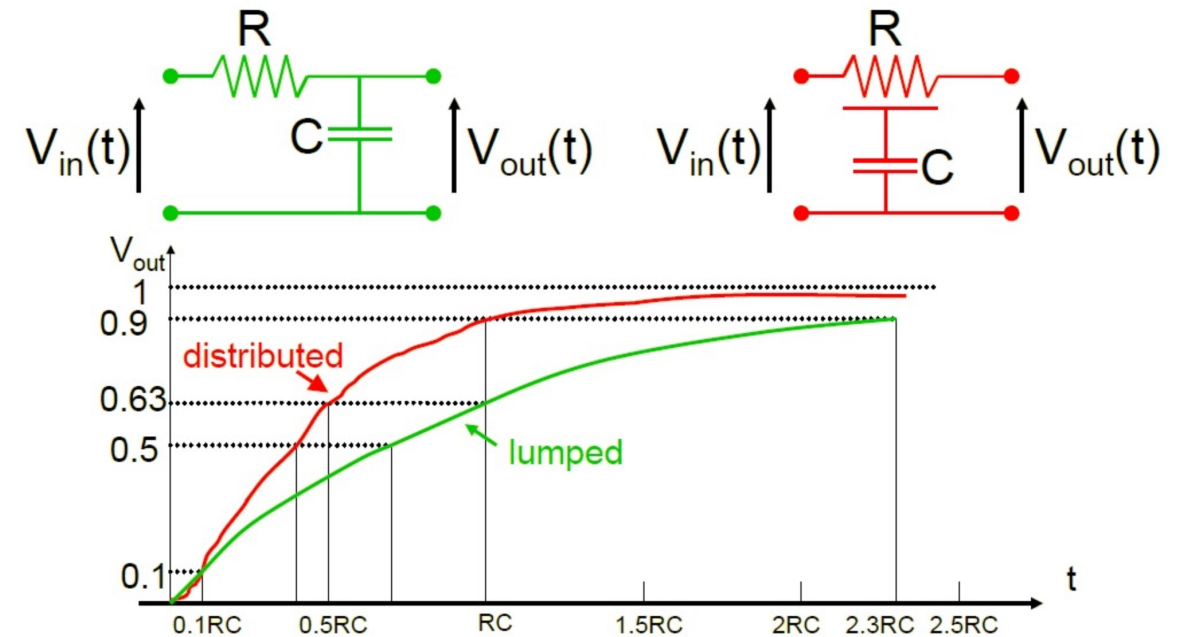
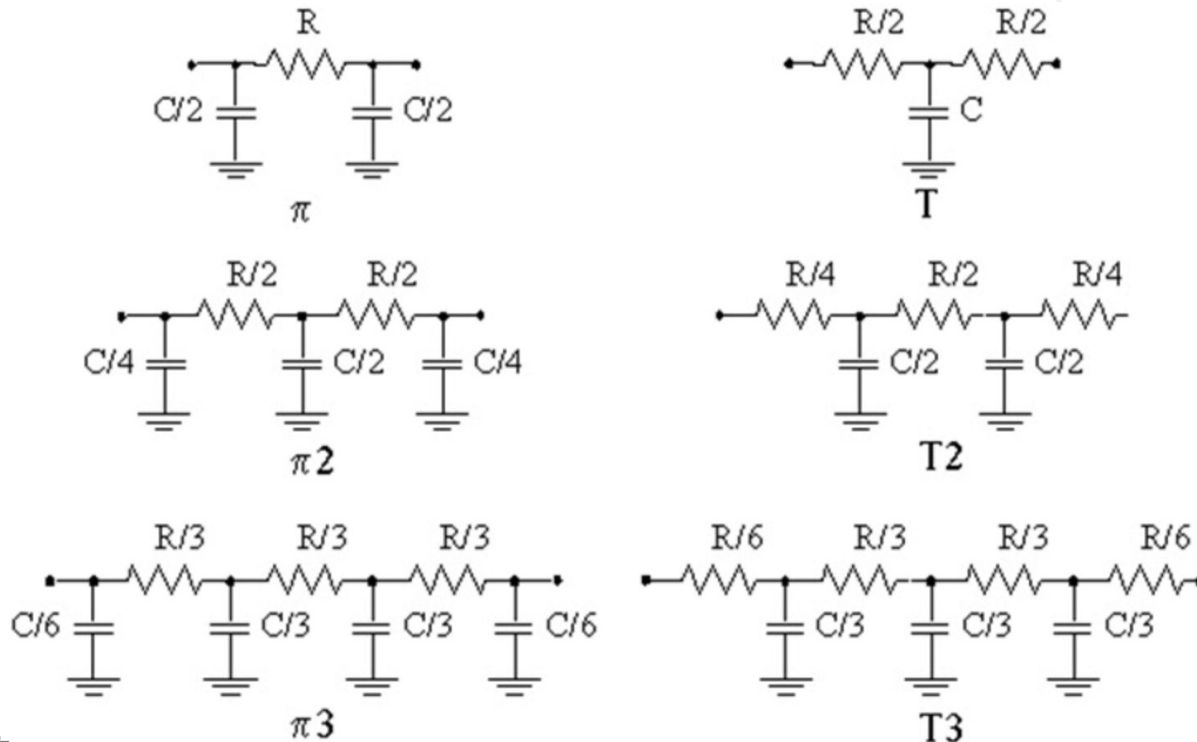


- Field Solver error stays within $\pm 2\%$ across all net capacitances
- Pattern matching LPE tools show error spreading to $\pm 20\%$, especially for small nets where pattern coverage is weakest
- Small nets are hardest to model accurately as fringe and coupling effects dominate and are most sensitive to pattern library gaps
- Pattern-based extractors trade accuracy for orders-of-magnitude faster runtime, which is acceptable for full-chip sign-off, not for model characterization

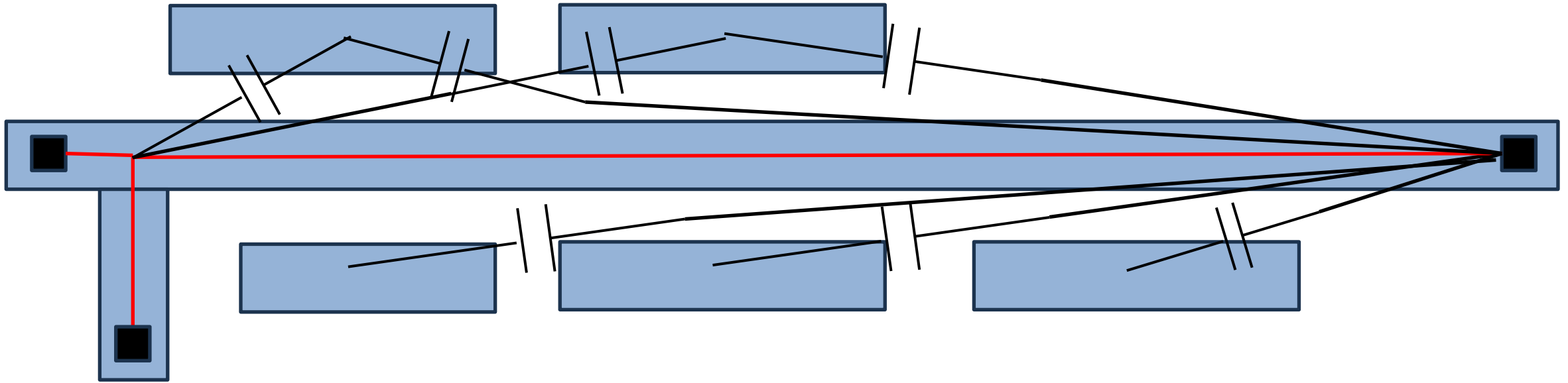
RC Distribution Impacts Timing Accuracy



How you model your RC network has an impact on wire delay calculation accuracy. Tradeoff is compute time vs. accuracy. More accuracy requires more data and more computation.

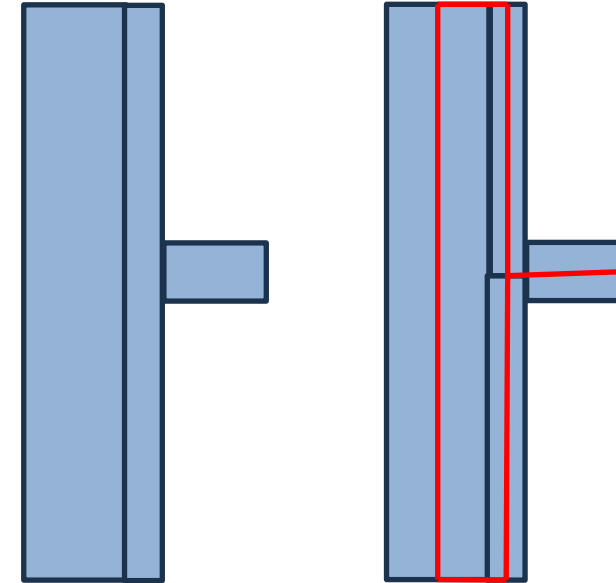
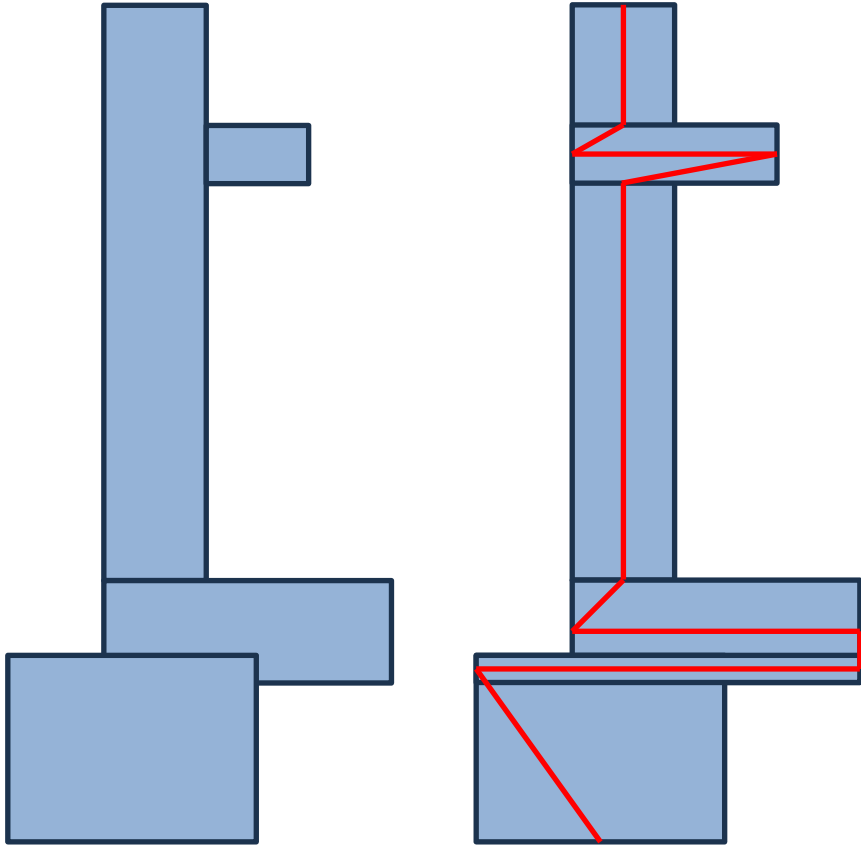


R Fracturing and C Distribution Matter



- How resistance fracturing is implemented and how coupling capacitances are attached to resistance subnodes makes a difference on RC delay accuracy.
- But excessive fracturing points creates more data and more computation time for delay analysis.
- Function usually controlled by a simple parameter (d or r).

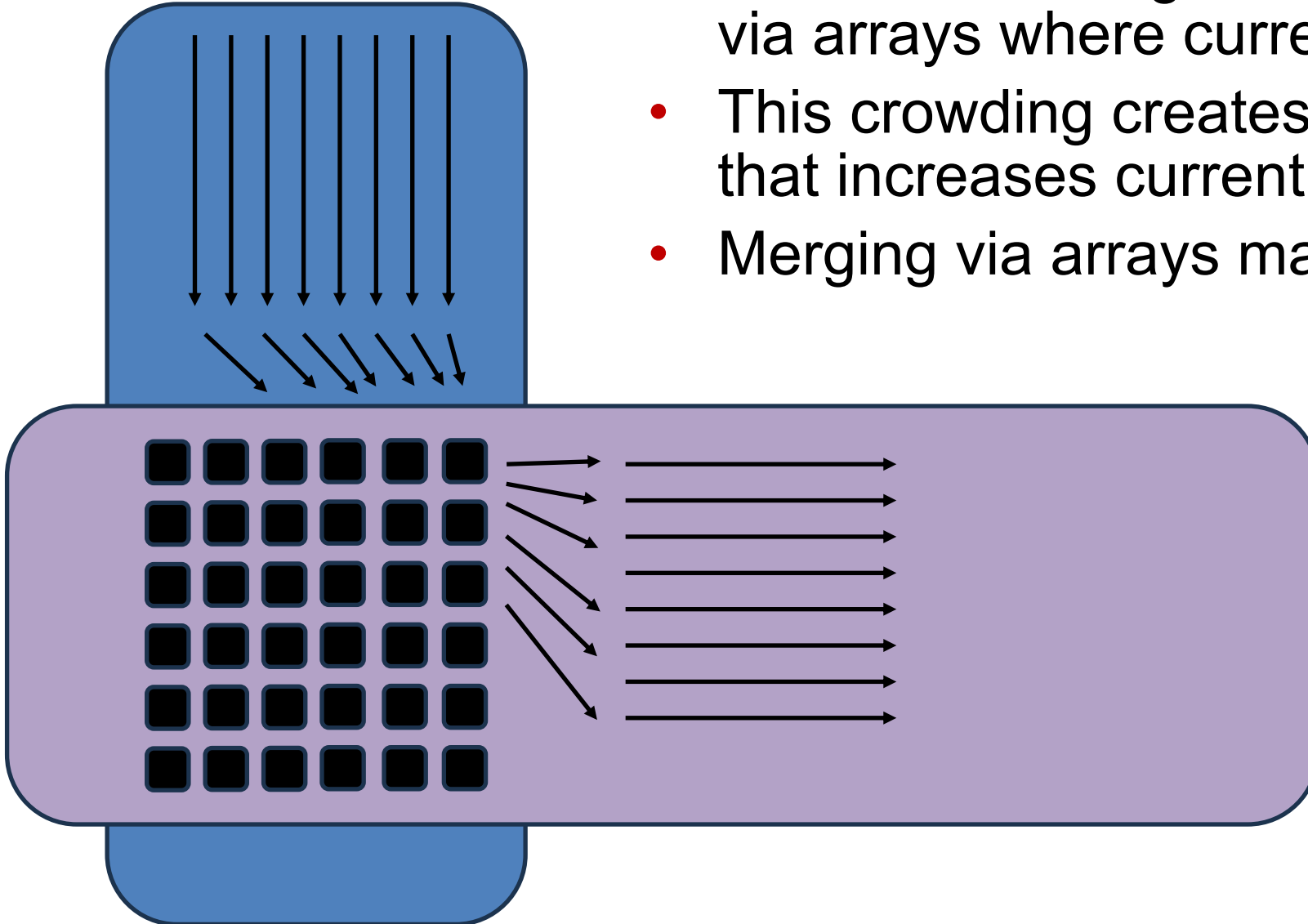
Resistance Fracturing – Errors



- When fracturing for resistance, geometries must be merged based on current flow and fractured appropriately.
- Assuming wrong flow or incomplete merging creates errors and/or loops.

Current Crowds in Corners and Via Arrays

- Current crowding occurs in corners and in via arrays where current changes direction.
- This crowding creates nonuniform current that increases current density.
- Merging via arrays may miss EM issues



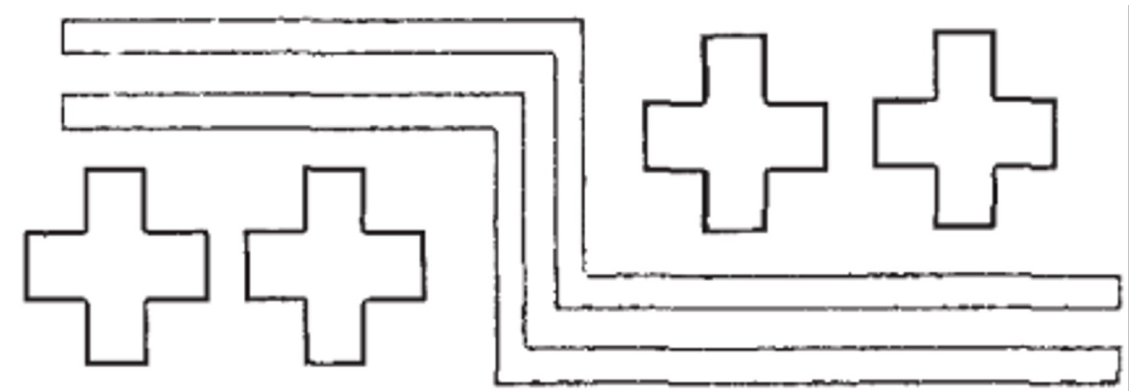
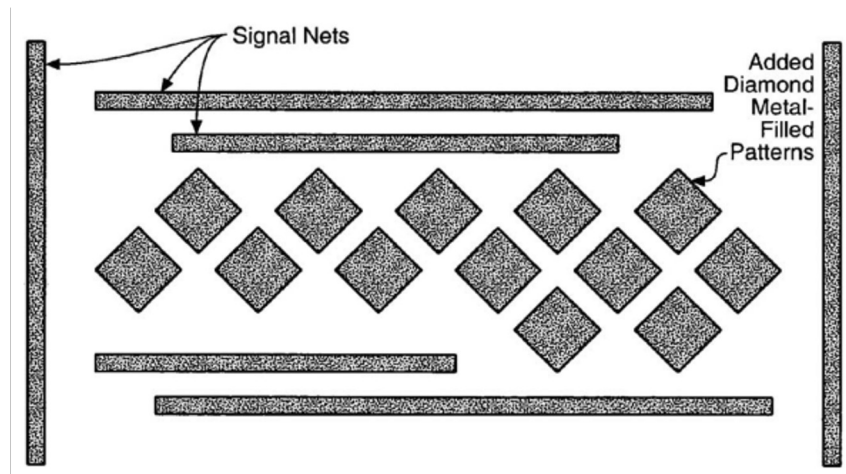
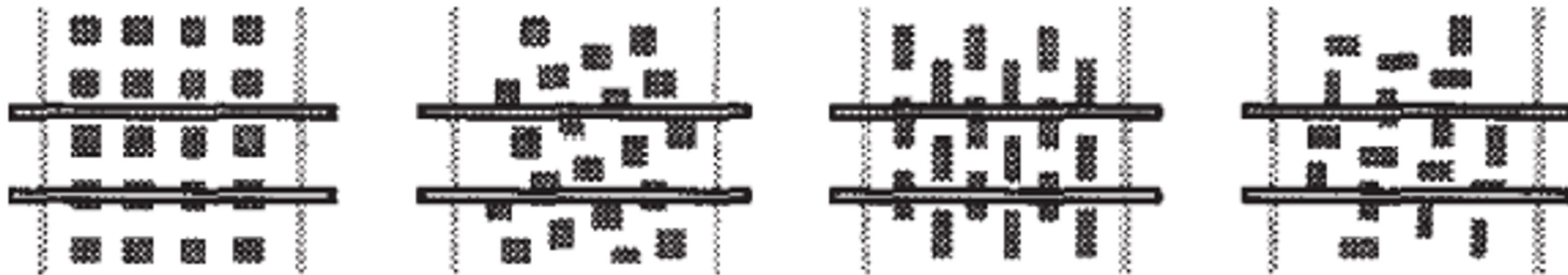
Open Issue: Metal Fill

- Extraction patterns up to 10um in size
- Metal density and fill impact is measured on a 50um grid
- Metal fill pattern and design is added after P&R is done
- Floating fill creates a large coupling capacitance matrix

- It is **very difficult** to integrate metal fill into extraction model characterization !
 - Add thickness parameter to be used at full chip extraction?
 - Add C matrix analysis to estimate effective wire cross coupling?
 - Explicit fill is most easily treated as grounded metal (OpenRCX)

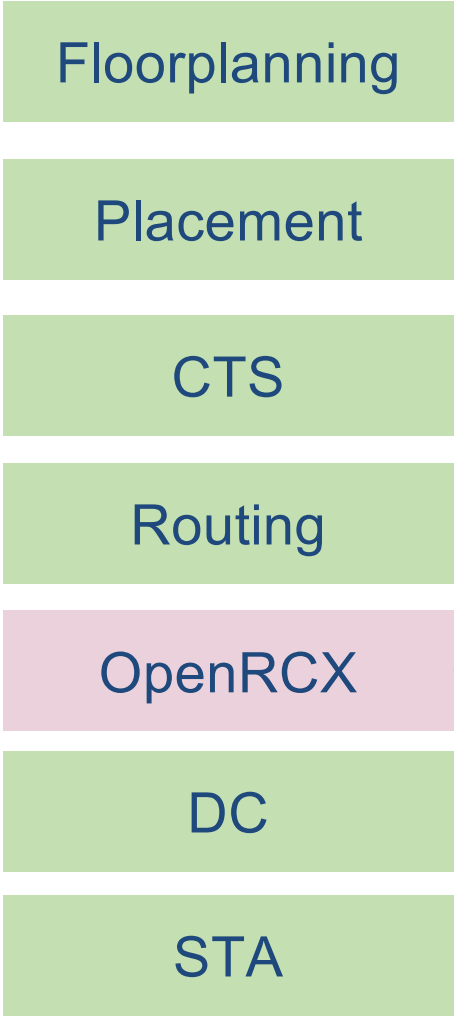
Other Examples <https://vlsicad.ucsd.edu/Publications/Journals/j84.pdf>

- Top (l-r): traditional, staggered, alternating, basket-weave why?
- Bottom: diamond and plus-sign patterns

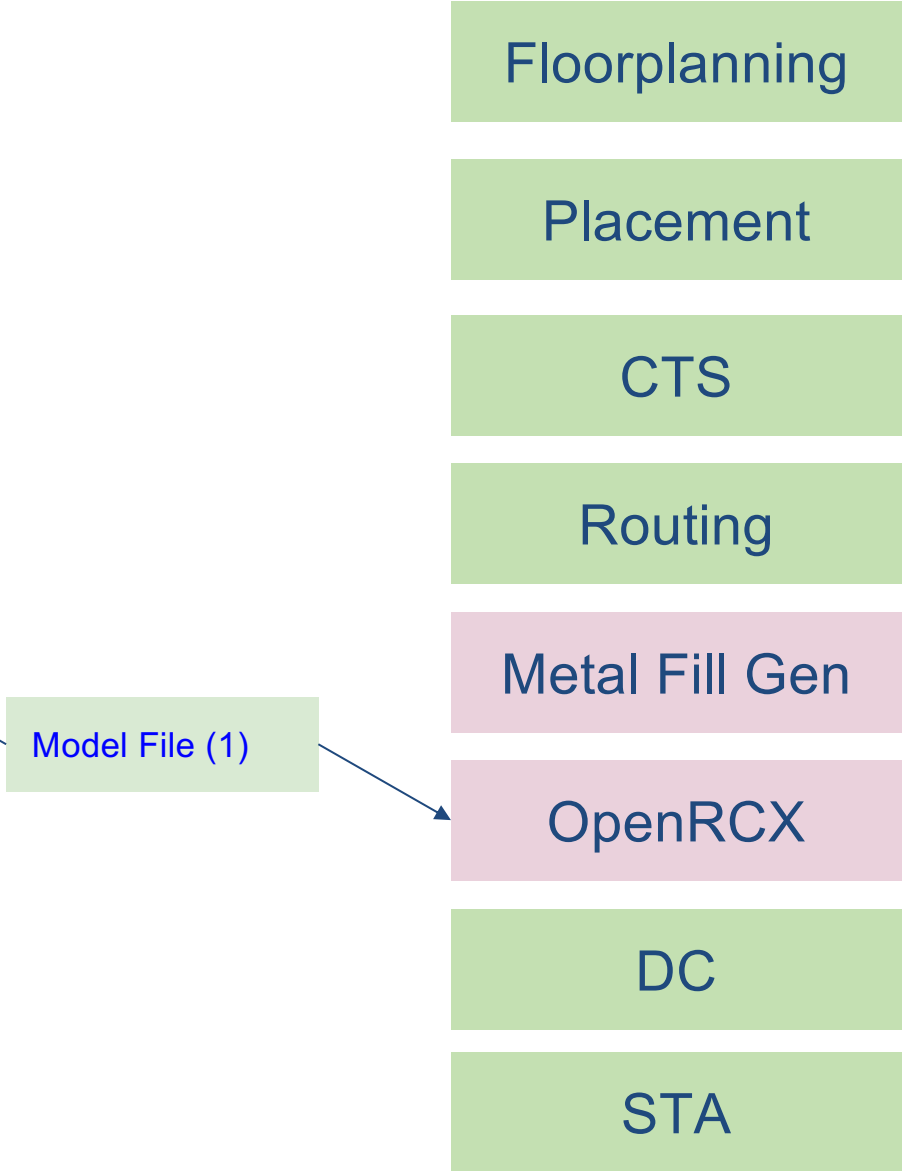


OpenRCX Flow

No Metal Fill Flow



Metal Fill Flow



If metal fill is present in the design, it will be extracted as grounded wires.

OpenRCX Internal Flow

Read Design (LEF, DEF, ODB)

Load routed geometry and net connectivity from the design database

Net Connectivity

Identify all nets and their associated wire segments and vias

Read Model File

Load characterized parasitic models for the target process corner

Geometry Fracturing for R

Break wire segments into sub-segments at junctions and direction changes to calculate R

Build Geometry Structures

Construct internal geometric representations of each fractured segment for pattern matching

Extract Resistance per Wire

Compute resistance for each fractured segment based on sheet resistance and geometry

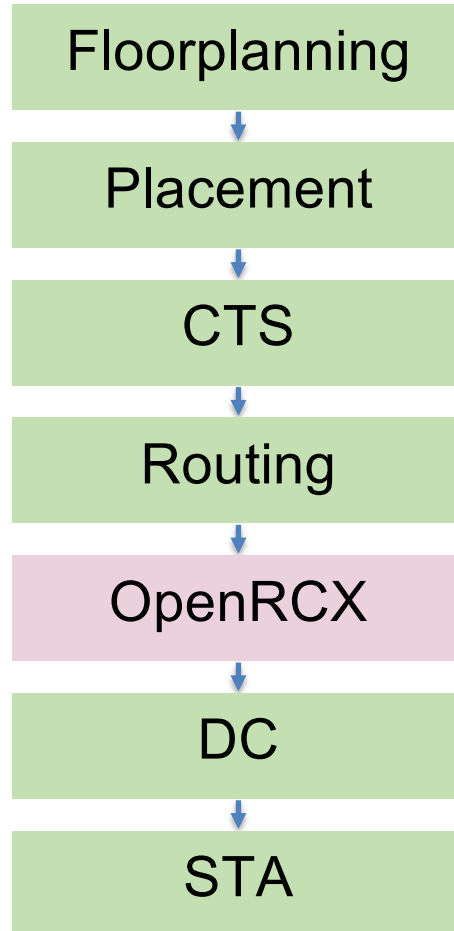
Extract Capacitance per Wire

Match each segment against pattern library to estimate coupling and grounded capacitance

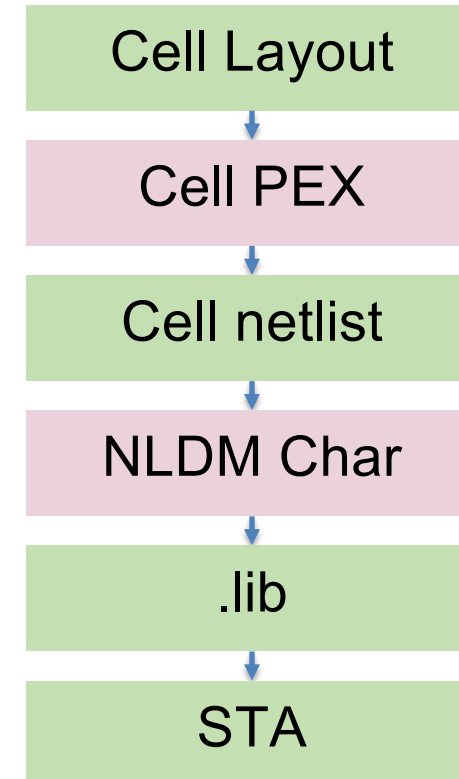
Aside — Open-Source EDA on the Cell Side

- Two parallel chains both feed STA
- **Chip-level** chain (with OpenRCX) runs for every design
- **Cell-level** chain runs once per PDK and is then reused for every design that uses that library

Chip-level (per design)

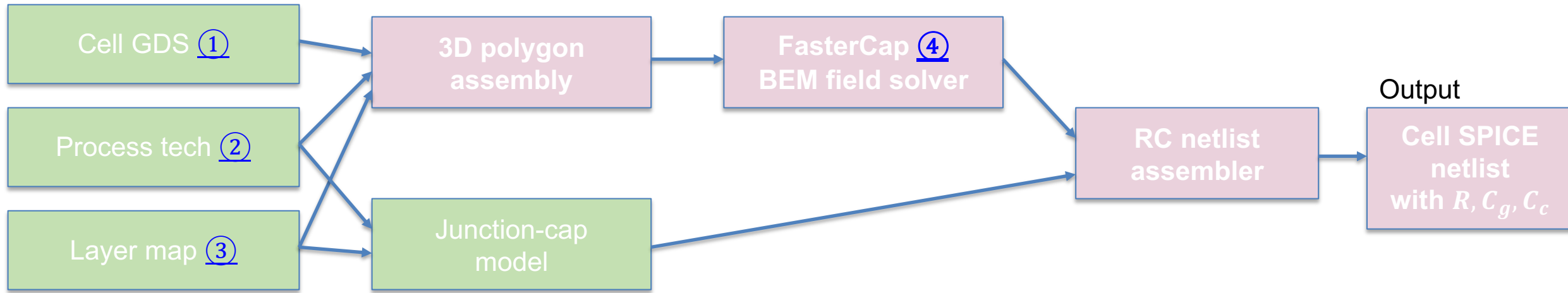


Cell-level (per PDK, once)



Cell PEX — Module Breakdown

Inputs



- Cell GDS / Process tech / Layer map — cell layout, vertical layer stack, and GDS-layer $\leftrightarrow$ name mapping
- 3D polygon assembly — extrudes the 2-D layout polygons into a **3-D conductor model**
- Junction-cap model — per-net source / drain **junction capacitance** derived from the same 3-D geometry
- FasterCap (BEM field solver) — solves the Maxwell capacitance matrix between every pair of conductors
- RC netlist assembler — wires the per-net R, C_g, C_c, C_j into a SPICE subcircuit for the cell

① ② ③ github.com/IHP-GmbH/IHP-Open-PDK (sg13g2_stdcell.gds / sg13g2_typ.itf / sg13g2.map)

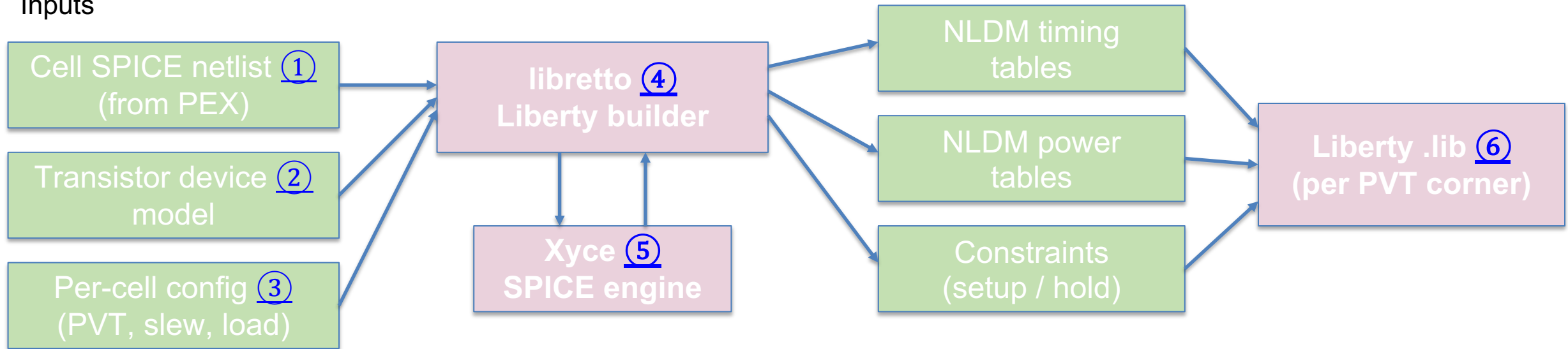
④ github.com/ediloren/FasterCap / github.com/george-goudroumanis/FasterCAP_v2

Library Characterization

- A standard-cell library (.lib) gives per-cell timing and power to synthesis, P&R, and STA
- Each cell: SPICE-measured under (input slew $\times$ output load), stored as a 2-D NLDM lookup table
- One .lib per PVT corner (process / voltage / temperature)
- STA needs both: SPEF (chip RCX) + .lib (library characterization)
- Commercial: Cadence Liberate, Synopsys SiliconSmart
- Open-source: libretto + Xyce (other open SPICE: ngspice)

Library Characterization — Module Breakdown

Inputs



- Cell SPICE netlist — the cell, ideally **with RC parasitics**
- Transistor device model — SPICE FinFET / BSIM model cards
- Per-cell config — PVT corner, slew × load grid, pin map
- libretto — drives characterization: builds SPICE testbenches, parses measurements, writes Liberty
- Xyce — open SPICE engine that libretto calls for each transient run
- NLDM tables — delay / transition / power / setup / hold lookup tables for every timing arc

① ② ⑥ github.com/IHP-GmbH/IHP-Open-PDK (sg13g2_stdcell.cdl / cornerMOSlv.lib / sg13g2_stdcell_typ_1p20V_25C.lib)
③ ④ github.com/snishizawa/libretto (gen_cmd_IHP130.py) ⑤ github.com/Xyce/Xyce

Library Characterization — Input / Output Files

1

sg13g2_stdcell.cdl

Stage: Xyce DUT — the cell circuit under test.

What: SPICE netlist; one .SUBCKT per cell with transistor W, L, and net connections.

Why: *libretto runs transient SPICE on each cell — it needs the topology to instantiate and drive it.*

```
.SUBCKT sg13g2_inv_1 Y A VDD VSS
MX1 Y A VSS VSS sg13_lv_nmos
    w=740n l=130n ng=1
MX0 Y A VDD VDD sg13_lv_pmos
    w=1.12u l=130n ng=1
.ENDS
```

2

cornerMOSlv.lib

Stage: co-loaded with the CDL at every Xyce (or ngspice) run.

What: PSP 103.6 model; corner-scaled I–V and C parameters for sg13_lv_nmos / pmos.

Why: *CDL gives topology; model gives physics. Both are required to solve any operating point.*

```
.LIB mos_tt
.param sg13g2_lv_nmos_ctl = 1.2080
.param sg13g2_lv_nmos_muew = 0.8500
.param sg13g2_lv_nmos_toxo = 1.0000
... (50+ PSP parameters)
.ENDL
```

3

gen_cmd_IHP130.py
→ .cmd (or template)

Stage: setup — generates one .cmd file per cell before simulation.

What: maps cell → pin names, Boolean function, input-slew (ns) and output-load/cap (pF) sweep vectors, file paths.

Why: *libretto is PDK-agnostic; gen_cmd_IHP130.py automates the mapping for all IHP-SG13G2 cells.*

```
gen_comb('sg13g2_inv_1', 'INV',
inputs=['A'], outputs=['Y'],
func=['Y=!A'],
slew=[0.02,0.10,...], # ns
load=[0.001,0.023,...], # pF
cell='NETLIST/inv_1.spi')
```

6

sg13g2_stdcell.lib

Stage: output — read by synthesis (Genus/Yosys) and STA (PrimeTime/OpenSTA).

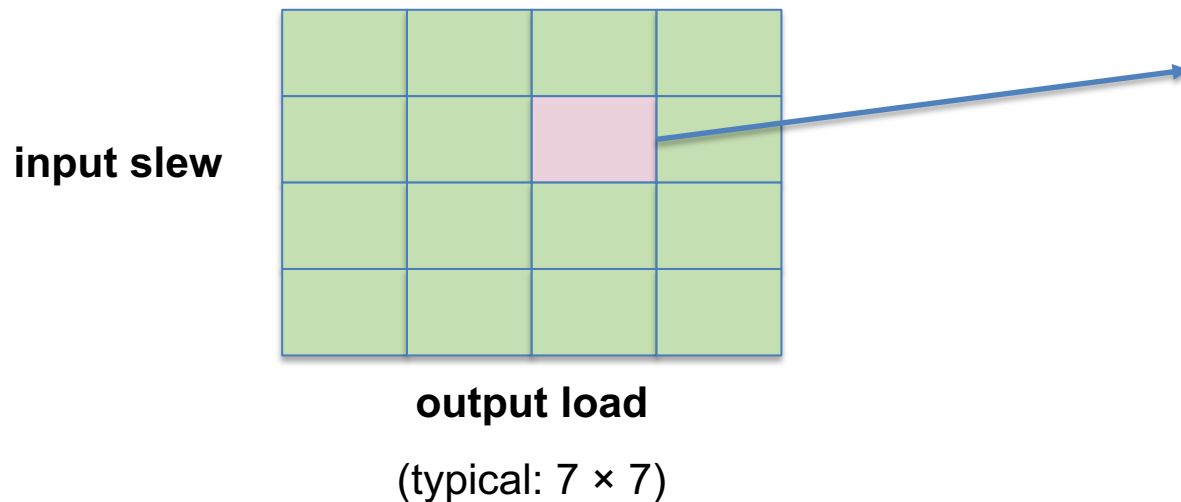
What: Liberty NLDM; 7×7 delay / slew / power tables per timing arc at TT 1.2 V 25 °C.

Why: *synthesis never runs SPICE — it looks up pre-computed tables. This file bridges simulation to timing closure.*

```
cell (sg13g2_inv_1) {
  cell_rise (7×7 slew × load) {
    index_1 ("0.019 0.097 ..."); # slew
    index_2 ("0.001 0.023 ..."); # load
    values ("0.021 0.085 ...",
            "0.040 0.127 ...", ...); } }
```

NLDM Lookup Table — What libretto Actually Builds

For one cell, one timing arc $\rightarrow$ one 2-D table. Each table entry is a separate SPICE measurement.



One SPICE run

- **libretto** builds the testbench: PWL ramp at (input slew), (output load) at the output, non-measured inputs fixed to the enabling state
- **Xyce** runs the transient simulation
- **libretto** parses the waveform
 - delay = 50 % $\rightarrow$ 50 % (input $\rightarrow$ output)
 - transition = 10 % $\leftrightarrow$ 90 %
 - internal power = $\int V \cdot I$ over the 1 % – 99 % switching window

- One cell $\rightarrow$ many timing arcs (each input pin $\times$ each output transition $\rightarrow$ a separate table)
- One arc $\rightarrow$ several tables (delay, transition, internal power, setup / hold, ...)
- One PVT corner $\rightarrow$ one .lib that holds every table for every cell; many corners $\rightarrow$ many .lib files

Results — ASAP7 (TT 0.7 V 25 °C)

Field-solver PEX vs. Siemens Calibre xACT — 5 cells (INV, NAND2, NOR2, NAND3, NOR3)

Quantity	Ratio vs Calibre	What's still missing
Ground cap C _g (output, VDD)	0.92 – 1.04×	Graded doubly-gated PMOS pocket
Ground cap C _g (inner gate input)	1.12 – 1.52×	Gate ↔ floating-S/D coupling
Coupling cap C _c	0.71 – 1.10×	Gate device-coupling (Calibre XCALIBRATE)
Wire R (input nets)	0.69 – 0.73×	Gate-path interface contacts
Wire R (output Y)	0.91 – 1.07×	Fin-position-aware LISD model

Library characterization vs. ASAP7 reference .lib (202 RVT cells)

Cell group	Count	vs reference	What's still missing
Simple combinational (INV / BUF / NAND / NOR / AND / OR)	~100	0.985 – 0.999×	
XOR, XNOR	8	0.990 – 0.995×	
DFF, Scan DFF	17	within ±5%	
AOI / OAI large fan-in	~30	1.16 – 1.40×	Complex multi-input timing
ICG large drive	9	0.60 – 0.74×	Clock-gating cell delay
Latch D→Q, async DFF	7	arc missing	Transparent path, async arcs

OpenRCX Future Wish List

- Creating a field solver suitable for characterizing patterns without the use of commercial “qualified” extractors?
- Developing a better solution for accounting for metal fill effects?
- Developing next generation models using the new highly parameterized patterns?
 - AI research opportunity?
- Developing a new GPU-optimized engine?

OpenRCX: How to Run

Docs: <https://github.com/The-OpenROAD-Project/OpenROAD/tree/master/src/rcx/doc>

By
Dimitris Fotakis

6/5/2025

How to Run RCX on a design - Using Standard Files

Tech.LEF

Macro.LEF

Design.DEF

Model File (*)

```
read_lef $lef_tech
read_lef $lef
read_def $def
```

```
define_rcx_corners -corner_list "max typ min"
```

```
extract_parasitics -ext_model_file model_file -version 2.0
write_spef 3corners.spef
```

OpenROAD
(OpenRCX)

design.SPEF

(*) Model File is generated once per process node;
See detailed instructions on how to do model
generation:

https://github.com/The-OpenROAD-Project/OpenROAD/blob/master/src/rcx/doc/def_model_generation.README

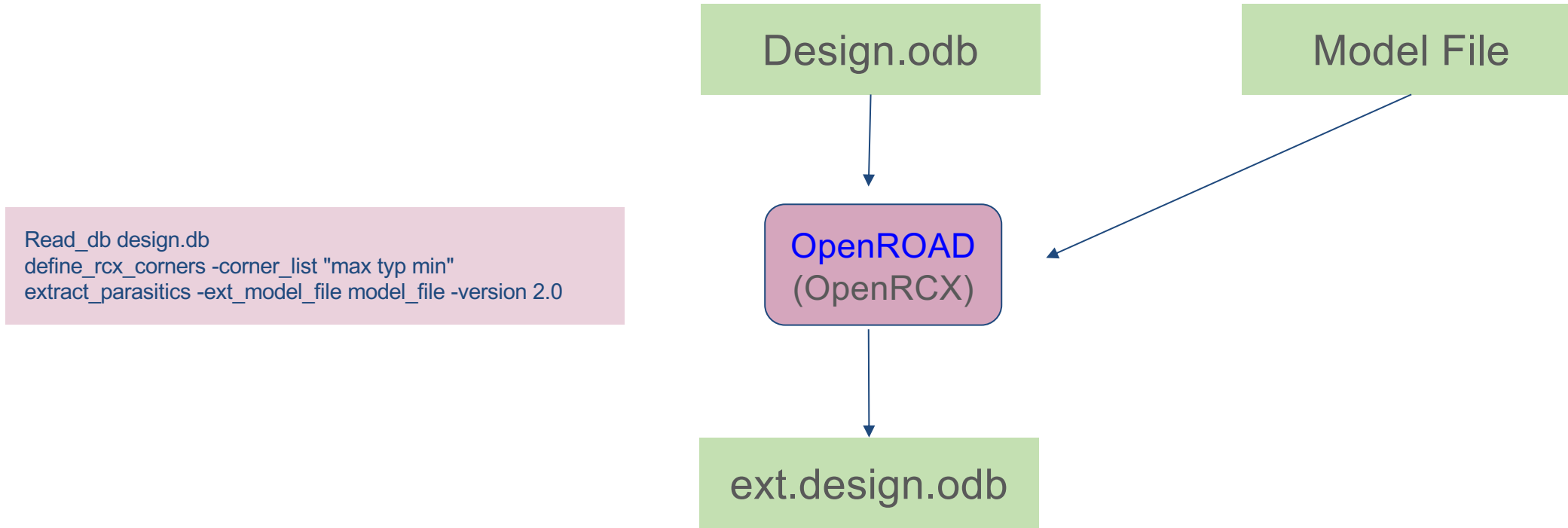
https://github.com/The-OpenROAD-Project/OpenROAD/blob/master/src/rcx/doc/FasterCap_model_generation.README

Details and OpenROAD Tcl Commands:

https://github.com/The-OpenROAD-Project/OpenROAD/blob/master/src/rcx/doc/run_extraction.README

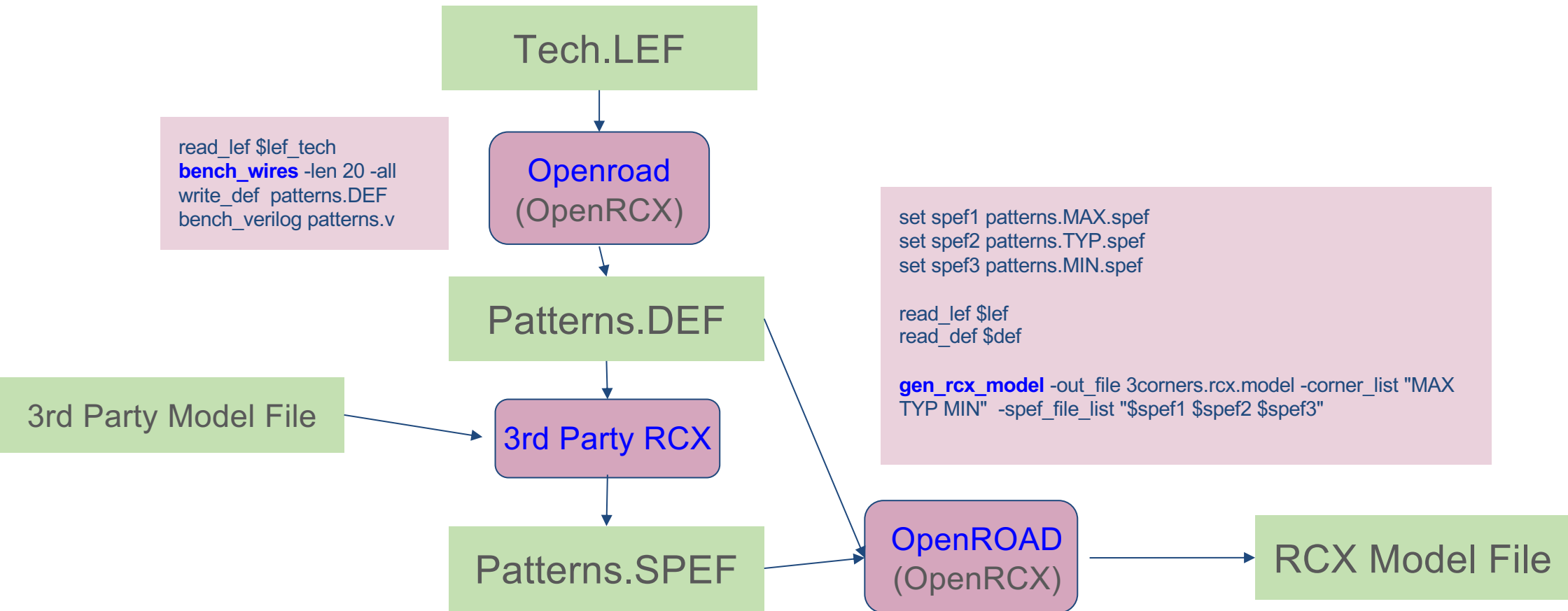
Note: OpenRCX requires the same files as any other commercial extractor

How to Run RCX on a design - Using odb



OpenRCX Model Generation: Option1 - DEF Based (User Flow)

https://github.com/The-OpenROAD-Project/OpenROAD/blob/master/src/rcx/doc/def_model_generation.README



OpenRCX Model Generation: Option2 - FasterCap Based (OS Flow)

https://github.com/The-OpenROAD-Project/OpenROAD/blob/master/src/rcx/doc/FasterCap_model_generation.README

Scripts to run the pieces of this flow: https://github.com/The-OpenROAD-Project/OpenROAD/tree/master/src/rcx/rule_scripts

